

CINEIQ PROJECT REPORT

Sentiment-Aware Hybrid AI Movie Recommendation Model

by Athulkrishna.k

Contents

1	The Concept	2
2	System Architecture	2
2.1	The Backend Engine (FastAPI)	2
2.2	The Frontend Interface (Streamlit)	2
3	The Engine	2
3.1	Hybrid Mathematical Filtering (TF-IDF & TruncatedSVD)	3
3.2	Real-Time Sentiment Modification (DistilBERT)	3
3.3	Explainable AI (XAI) via LIME	3
4	Data Engineering & Optimization	3
4.1	Data Extraction & Transformation	4
4.2	Storage and Memory Optimization	4
5	Systemic Challenges & Resolutions	4
5.1	Resolving Matrix-to-DataFrame Index Misalignment	4
5.2	Preventing PyTorch Tensor Token Overflows	4
6	Conclusion and Future Scope	5

1. The Concept

The premise of this project is to build an ML model that recommends the best possible movies for the user to watch next based on a movie that they love. The engine factors user sentiment, ratings (collaborative filtering), and keyword importance (TF-IDF, content-based filtering) to spew out recommendations.

2. System Architecture

2.1. The Backend Engine (FastAPI)

Acting as the “Brain” of the application, the backend is a high-performance REST API powered by FastAPI. It hosts the core machine learning models in local memory, including the pre-computed Scikit-Learn hybrid matrices (TF-IDF for content and TruncatedSVD for collaborative filtering). When a recommendation request is received, the backend executes the mathematical similarity search and simultaneously triggers a live web scraper to fetch current user reviews for the target movies. These reviews are processed through a DistilBERT model to calculate a real-time sentiment re-ranker, penalizing mathematically similar movies that have poor modern reception. Finally, the backend utilizes LIME (Local Interpretable Model-agnostic Explanations) to package the algorithmic decisions into human-readable justifications before returning the JSON payload to the client.

2.2. The Frontend Interface (Streamlit)

Acting as the “Face” of the application, the frontend is an interactive, state-managed web application built with Streamlit. It handles all user inputs and communicates asynchronously with the FastAPI backend. To optimize performance and reduce server load, the frontend utilizes Streamlit’s memory caching to store active datasets, preventing redundant API calls during a single session. Beyond the primary recommendation search engine, the frontend houses a “User Taste Profiler”—an analytics dashboard leveraging the Plotly library to visualize cinematic DNA through interactive data visualizations, including Genre Radar charts and Era/top directors/actors Histograms.

3. The Engine

The core recommendation engine of CINEIQ stays away from traditional, single-algorithm models by employing a multi-layered hybrid approach. It sequences mathematical similarity matrices with live Natural Language Processing (NLP) to generate optimum results.

3.1. Hybrid Mathematical Filtering (TF-IDF & TruncatedSVD)

The foundational layer of the engine computes baseline similarity scores by fusing two distinct recommendation strategies:

- **Content-Based Filtering:** The engine utilizes a TF-IDF (Term Frequency-Inverse Document Frequency) Vectorizer to parse the textual metadata of movies, specifically genres, overviews, and keywords. This transforms the descriptive text into a high-dimensional mathematical space, allowing the system to calculate cosine similarity scores between the structural plots of different movies.
- **Collaborative Filtering:** To account for user taste, the system applies TruncatedSVD (Singular Value Decomposition) to a massive user-item rating matrix. By reducing the dimensionality of millions of historical user ratings, the engine isolates latent taste profiles, identifying films that are frequently highly rated by users with similar watch histories.

3.2. Real-Time Sentiment Modification (DistilBERT)

While the hybrid mathematical model identifies films that are structurally and historically similar, it cannot account for how a movie has aged or its current cultural reception. To solve this, CINEIQ integrates a dynamic NLP pipeline. When the baseline mathematical model outputs a list of candidate movies, the backend triggers a live web scraper to pull the most recent user reviews for those specific films from the internet.

These text reviews are truncated to respect token limits and passed through a pre-trained HuggingFace DistilBERT pipeline fine-tuned for sentiment analysis. The NLP model assigns a positive or negative probability weight to the current discourse surrounding the film. This sentiment score acts as a mathematical multiplier, physically penalizing mathematically similar movies that are suffering from poor modern reception, and boosting hidden gems that are actively trending positively.

3.3. Explainable AI (XAI) via LIME

A significant flaw in modern Deep Learning recommenders is the “Black Box” effect, where users are given outputs without understanding the algorithmic reasoning. To ensure complete transparency, CINEIQ implements LIME (Local Interpretable Model-agnostic Explanations). When a final recommendation is served to the frontend, the LIME module highlights the specific textual features and plot vectors that drove the mathematical similarity, providing the user with a human-readable justification for the AI’s decision.

4. Data Engineering & Optimization

4.1. Data Extraction & Transformation

The initial dataset consisted of massive, multi-gigabyte Kaggle files containing deeply nested JSON arrays for movie genres, production companies, and spoken languages. Custom Python scripts were engineered to parse the stringified dictionaries using the Abstract Syntax Tree (`ast`) library, extracting only the relevant categorical vectors.

4.2. Storage and Memory Optimization

To bypass strict repository file-size limits (such as GitHub's 25MB standard constraint) without compromising the dataset's integrity, the final `clean_masterdata.csv` was natively compressed into a `.zip` archive.

5. Systemic Challenges & Resolutions

Building a decoupled, multi-model microservice introduced several complex engineering challenges, primarily involving memory alignment and tensor limitations.

5.1. Resolving Matrix-to-DataFrame Index Misalignment

The Problem: Dropping duplicate rows and null values created fragmented numerical gaps in the Pandas DataFrame index. When the Scikit-Learn `TfidfVectorizer` built the numerical similarity matrix, it utilized a strict 0-to-N positional index. Because the Pandas index and the Matrix index no longer aligned, the backend began returning mathematically incorrect movie recommendations (e.g., recommending a children's movie when an action movie was mathematically selected).

The Solution: A strict `.reset_index(drop=True)` enforcement was applied immediately after data cleaning and just before matrix compilation. This ensured that the DataFrame's physical row numbers perfectly mapped to the vectorizer's matrix coordinates, restoring 100% accuracy to the algorithm's outputs.

5.2. Preventing PyTorch Tensor Token Overflows

The Problem: The HuggingFace DistilBERT model utilized for the real-time sentiment analysis has a strict maximum input limit of 512 tokens (roughly 350-400 words). When the backend scraped exceptionally long, essay-style movie reviews from the internet, it triggered a PyTorch tensor overflow, causing the FastAPI server to crash entirely.

The Solution: A programmatic truncation safeguard was integrated into the NLP pipeline. Before being tokenized and passed into the neural network, all scraped internet strings are automatically truncated to a safe character limit.

6. Conclusion and Future Scope

CINEIQ successfully demonstrates a modern approach to recommendation engines by moving beyond static collaborative filtering. By fusing deep mathematical matrix similarities with live, contextual NLP sentiment analysis, the application generates highly accurate, culturally relevant, and fully explainable AI recommendations.

Future iterations of the system may focus on expanding the architecture for a larger user base. Proposed updates include migrating the static CSV database to a fully managed cloud SQL instance (such as AWS RDS), implementing user authentication to allow for persistent session tracking across multiple logins, and integrating multimedia APIs to stream movie trailers directly within the Streamlit frontend.